# CAPSTONE PROJECT REGISTER

**Class:** **Duration time:** from ...../20.... To ...../20.....

**(*) Profession:** <Software Engineer> **Specialty:** <ES> ☐ <IS> ☐ <JS> ☐

**(*) Kinds of person make registers:** Lecturer ☐ Students ☑

## 1. Register information for supervisor (if have)

| No. | Fullname | Phone | E-Mail | Title |
|---|---|---|---|---|
| Supervisor | Lại Đức Hùng | | Hungld5@fe.edu.vn | Mr. |
| Supervisor | | | | Mr. |

## 2. Register information for students (if have)

| Full name | Student code | Phone | E-mail | Role in Group |
|---|---|---|---|---|
| | | | | Leader |
| | | | | Member |
| | | | | Member |
| | | | | Member |

## 3. Register content of Capstone Project

### (*) 3.1. Capstone Project name:

- **English:** PickleVerse – Multiplayer Pickleball Skill-Training and Match Game with AI Coaching
- **Vietnamese:** PickleVerse – Game Huấn Luyện Kỹ Thuật và Thi Đấu Pickleball Nhiều Người Chơi Tích Hợp AI
- **Abbreviation:** PKV

**a. Context:**

Pickleball has become one of the fastest-growing sports in Vietnam since 2024. New courts have appeared rapidly across Ho Chi Minh City, Hanoi, and Da Nang, and the sport now attracts a very broad demographic – office workers, university students, and older adults – because it is easy to start yet technically deep.

However, most newcomers learn by imitation on the court with no structured feedback. Qualified coaches are scarce, and private lessons are expensive. As a result, beginners frequently reinforce incorrect fundamentals – serve motion, dink control, the “third-shot drop,” and the non-volley-zone (“kitchen”) rules – that are difficult to unlearn later. In addition, casual players often struggle to find partners at a matching skill level, and there is little way to practice fundamentals when a physical court is unavailable due to weather, cost, or scheduling.

A game-based platform can close both gaps. It lets players learn and drill core techniques through interactive lessons and mini-games, then apply them in online matches against friends or AI opponents. Modern game engines and lightweight machine-learning models make it feasible to give near real-time technique feedback based on in-game play data and to provide adaptive AI opponents, turning repetitive fundamentals practice into an engaging, social experience.

**Target users / customers:** primary users are Vietnamese pickleball beginners and casual players; potential B2B customers include pickleball clubs, sports academies, and community sport centers seeking an affordable digital training and community tool.

**b. Proposed Solutions**

- Build a cross-platform 3D Pickleball game (Unity) offering solo training, online multiplayer matches, and practice against adaptive AI opponents
- Design a structured lesson-and-drill system teaching core techniques (serve, dink, volley, third-shot drop, kitchen/NVZ rules) with in-game scoring and progression
- Implement an authoritative real-time multiplayer server enabling 1v1 (singles) and 2v2 (doubles) matches with friends and skill-based matchmaking
- Provide a basic AI layer: adaptive bot opponents, an Elo-style skill-rating and matchmaking system, and a technique-feedback engine that analyzes in-game telemetry to detect common mistakes and suggest corrections
- Deliver an admin web dashboard for user management, lesson/content management, live match monitoring, moderation, and analytics

**c. Functional Requirements**

**Player Game Client (Unity – Desktop & Mobile)**

- Register/login, create a player profile, and complete a self-assessment during onboarding
- Interactive tutorial teaching pickleball rules and the game control scheme
- Training mode: structured lessons per technique (serve, dink, volley, third-shot drop, court positioning) with guided drills and mini-games
- Star-based scoring (1–3) per drill; XP and level progression; unlock new lessons and cosmetic rewards
- Practice vs AI: play points and games against an adaptive bot with selectable difficulty
- Online multiplayer: 1v1 singles and 2v2 doubles; play with friends via room code or quick-match matchmaking
- Near real-time technique feedback after drills and matches (accuracy, timing, shot-selection tips)
- Personal progress dashboard: accuracy trends, practice time, technique mastery, achievement badges
- Friends list, match history, and leaderboard ranking
- Daily challenges with streak rewards

**Basic AI Features**

- **Adaptive AI opponent:** a behavior-tree bot whose shot selection, aggressiveness, and error rate scale to the player's current skill rating
- **Skill rating & matchmaking:** an Elo-style rating updated after each match; matchmaking pairs players of similar rating; a lightweight classifier estimates a new player's starting tier from onboarding and first sessions
- **Technique-feedback engine:** analyzes per-shot in-game telemetry (timing windows, contact position, shot accuracy, footwork/positioning metrics) to classify common errors (e.g., late serve, dink too high, stepping into the kitchen) and generate targeted tips
- **Personalized drill recommendation:** suggests the next drills based on the player's detected weak techniques

**Admin Web Dashboard**

- Admin login with role-based access control (player / admin roles)
- User management: search accounts, suspend/ban, reset progress, assign roles
- Content management: create/edit lessons, drills, difficulty parameters, and reference technique data; publish/unpublish
- Match & session monitoring: view active matches and completed match records; handle disconnect and abuse reports

- Analytics dashboard: active users, retention, popular lessons, technique-error heatmaps, matchmaking health
- Moderation: review reported players and behavior, resolve appeals

**d. Non-functional Requirements**

- Real-time multiplayer latency: server state sync targeting < 100 ms round-trip on typical Vietnamese broadband; client-side prediction and server reconciliation for smooth play
- Game frame rate: stable 60 FPS on mid-range devices
- AI technique-feedback processing: results returned within 1 second after a drill or point
- Scalability: backend supports 2,000 concurrent players and 500 concurrent live matches
- Availability: 99.5% uptime for core online services
- Compatibility: Windows 10+, Android 9.0+, iOS 14.0+ (Unity cross-platform export)
- Security: JWT authentication, data encrypted at rest (AES-256), server-authoritative match logic to prevent cheating
- Offline mode: training and drills playable offline; progress syncs on reconnect

**(*) 3.2. Main proposal content (including result and product)**

**a. Theory and practice (document):**

Students apply iterative agile development with UML 2.0 system modeling. Documentation deliverables include: User Requirements Specification, Software Requirements Specification (SRS), System Architecture Design, Database Schema, Multiplayer Netcode Design, AI Module Design, Testing Strategy & Results, User Manual, Administrator Guide, API Documentation, complete source code repositories, and deployable software packages.

*Game Engine & Client:*

- Unity (LTS) with C# for 3D gameplay and physics (ball trajectory, bounce, collision)
- Multiplayer: Unity Netcode for GameObjects with a dedicated (headless) server build; Mirror or Photon Fusion as alternatives; server-authoritative model
- UI: Unity UI Toolkit; input via Unity Input System

*Server-side Technologies:*

- Backend API: Spring Boot (Java) or Python FastAPI – authentication, profiles, progression, matchmaking, leaderboards, content
- Real-time game servers: dedicated Unity headless server instances orchestrated by a matchmaking service; Redis for the matchmaking queue and session state
- AI service: Python FastAPI microservice – scikit-learn / lightweight models for skill classification and technique-error classification; behavior-tree logic for the bot; Elo for rating
- Database: PostgreSQL (users, matches, lessons, progress, telemetry) + Redis (queue, sessions, leaderboard cache)
- Cloud storage: AWS S3 / GCP Cloud Storage for assets and match replays
- Deployment: Docker containers; GCP Cloud Run / AWS EC2 for services; a VM/container fleet for game servers

*Client-side Technologies (Admin Web):*

- Web framework: ReactJS + Vite; Tailwind CSS + shadcn/ui component library
- Server state: React Query; charts: Recharts / Chart.js for analytics

**b. Products:**

- **PickleVerse Game Client (Desktop + Mobile):** Unity game with training mode, AI practice, and online multiplayer
- **Backend API Service:** REST endpoints for authentication, progression, content, matchmaking, and leaderboards

- **Real-time Multiplayer Server:** authoritative dedicated game server plus a matchmaking service
- **AI Service:** skill-rating/matchmaking, adaptive bot, and technique-feedback engine
- **Admin Web Dashboard:** user, content, match, and analytics management
- **Technical Documentation:** architecture diagrams, netcode and AI design specs, API docs, database schema, user and admin manuals, deployment guides

**c. Proposed Tasks:**

- **Package 1 (Week 1–2):** Foundation – Unity project + ball/court physics prototype, backend scaffolding (API + DB schema), JWT authentication, CI/CD pipeline, single-player core court mechanics
- **Package 2 (Week 3–4):** Training system – lesson/drill framework, technique mini-games (serve, dink, volley), scoring, XP/progression, profile dashboard
- **Package 3 (Week 5–7):** Multiplayer core – Netcode integration, authoritative server, 1v1 match flow, room code + basic matchmaking, client prediction/reconciliation, match history
- **Package 4 (Week 8–9):** AI layer – adaptive bot (behavior tree + difficulty scaling), Elo rating + matchmaking service, telemetry capture pipeline
- **Package 5 (Week 10–11):** AI technique feedback + doubles – technique-error classifier, feedback UI, drill recommendations; 2v2 doubles multiplayer; leaderboards and daily challenges
- **Package 6 (Week 12–13):** Admin web dashboard – user/content management, match monitoring, analytics; end-to-end integration testing; gameplay balancing
- **Package 7 (Week 14):** Deployment & documentation – production deployment, beta test with real players, bug fixes, complete documentation bundle, final presentation

## 4. Other comments (propose all relative things if have).

- **SE complexity justification:** the authoritative real-time multiplayer netcode (client prediction, server reconciliation, anti-cheat) is the core engineering challenge that distinguishes this project from a single-player game; combined with a service-oriented backend, an AI microservice, and an admin web application, the system demonstrates a full multi-tier architecture.
- **AI scope kept deliberately foundational:** a behavior-tree bot, Elo rating, and lightweight telemetry classifiers – not computer vision on real video – which is honest and feasible for a 4-student team in 14 weeks.
- **Scope discipline:** the initial release is limited to 1v1 and 2v2 on a single court type, about 5 core techniques and around 15 drills; ranked ladders, tournaments, spectator mode, and real-video swing analysis are reserved for future phases.
- **Anti-cheat:** server-authoritative match logic – clients send inputs, the server validates and computes outcomes.
- **Vietnamese market grounding:** Vietnamese-language UI, local court culture, and (post-capstone) MoMo/VNPay for cosmetic purchases.
- **Post-capstone roadmap:** cosmetic marketplace with MoMo/VNPay, club and tournament systems, a dedicated coach portal, and AR/real-video swing analysis.

| | |
|---|---|
| **Supervisor (If have)**<br>*(Sign and full name)* | HCM, date ...../...../20.....<br>**On behalf of Registers**<br>*(Sign and full name)* |